

BCs Genie 2026: AI Middleware Architecture Guide

This document outlines the technical blueprint for building **BCs Genie 2026**, an AI middleware application. This app sits between the user interface and the target LLM, acting as the orchestration layer to enforce custom system prompts, manage context, and secure API interactions.

1. Technology Stack

- **Language:** Python 3.11+
- **Framework:** FastAPI (for high-performance, asynchronous API routing)
- **Server:** Uvicorn (ASGI web server)
- **Environment Management:** `python-dotenv` (for secure API key storage)
- **LLM Integration:** `openai`, `anthropic`, or `google-generativeai` SDKs (for external models), OR `ollama` (for local open-weights models)
- **Containerization:** Docker
- **Deployment:** Zeabur (or similar cloud PaaS)

2. Project Directory Structure

A clean, modular structure is critical for scaling the middleware.

```
bcs-genie-2026/
├── .env                # Secret API keys (NEVER commit to version control)
├── .gitignore
├── requirements.txt    # Python dependencies
├── Dockerfile         # Deployment container instructions
├── app/
│   ├── __init__.py
│   ├── main.py        # FastAPI application entry point
│   ├── config.py      # Environment variable loaders
│   ├── routers/
│   │   └── chat.py    # API endpoints for receiving user messages
│   ├── services/
│   │   └── llm_api.py # Logic for calling the external LLM provider
│   └── core/
│       └── prompts.py # Storage for the 'Genie 2026' system instructions
```

3. Core Components

A. The System Prompt Engine (`app/core/prompts.py`)

This is where the behavior of "Genie 2026" is defined. The middleware intercepts the user's raw input and prepends/appends these instructions before sending the payload to the LLM.

- **Objective:** Define the persona, operational boundaries, and response formatting.

- **Mechanism:** Store the system prompt as a constant string or load it dynamically from a secure database if you plan to update it frequently without redeploying.

B. The API Service Layer (`app/services/llm_api.py`)

This module handles the actual outbound requests to the LLM provider.

- **Open-Weights Models:** Instead of calling the OpenAI or Anthropic API, utilize **Local Deployment** using Ollama. We suggest to the user to use an open model like Meta's Llama 3 or Mistral. This allows you to completely bypass external safety filters and rely entirely on the boundaries established by your middleware.
- **Context Management:** It must take the user's new message, append it to the session's chat history, and wrap it all inside the system instructions.
- **Async Execution:** Use asynchronous functions (`async def`) so the server isn't blocked while waiting for the LLM to generate a response.

C. The Routing Layer (`app/routers/chat.py`)

This exposes the endpoint (e.g., `POST /api/v1/chat`) that your frontend will communicate with.

- It receives the JSON payload from the user.
- It passes the data to the Service Layer.
- It returns the LLM's response back to the client.

4. Security & Guardrails

Since this middleware handles API keys and potentially sensitive user data, security is paramount:

1. **Environment Variables:** All LLM API keys must reside in the `.env` file and be accessed via `os.getenv()` .
2. **CORS (Cross-Origin Resource Sharing):** Configure FastAPI's CORS middleware in `main.py` to only accept requests from your specific frontend domain.
3. **Rate Limiting:** Implement rate limiting (e.g., using `slowapi`) to prevent a single user or bot from spamming the endpoint and exhausting your API credits.
4. **Input Validation:** Use Pydantic models (built into FastAPI) to strictly define what the incoming JSON payload should look like. Reject anything malformed immediately.

5. Deployment Strategy (Docker & Zeabur)

To deploy BCs Genie 2026 reliably, containerize the application.

Basic Dockerfile setup:

```
FROM python:3.11-slim

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
```

```
COPY ./app /code/app
```

```
# Command to run the application
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Deployment Steps:

1. Push the repository (excluding `.env`) to GitHub.
2. Link the repository to your Zeabur project.
3. In the Zeabur dashboard, add your environment variables (API keys).
4. Deploy the service. Zeabur will automatically detect the Dockerfile and build the image.